

INF1022 - Analisadores Léxicos e Sintáticos

Um Parser geral para LLC: O algoritmo CYK

Edward Hermann Haeusler Vitor Pinheiro de Almeida

PUC-Rio



O Algoritmo de Parsing geral CYK¹

Um analisador sintático ascendente capaz de fazer parsing em qualquer linguagem livre de contexto.

- 1 Algoritmo determinístico, mas não é APND;
- 2 Complexidade de análise $\mathcal{O}(n^3)$;
- 3 Método de análise ascendente, e;
- 4 Utiliza-se de gramáticas em forma normal de Chomsky

Forma normal de Chomsky:

$$\begin{aligned} S &\longrightarrow A_1 A_2 \mid a \\ A_i &\longrightarrow A_k A_j \mid a \end{aligned}$$

Não possui regras ϵ

¹Cocke Younger Kasami, 1961, 1965, 1970, etc

Algoritmo CYK (I)

Para facilitar a descrição do algoritmo, utiliza-se a entrada com um formato especial:

A cadeia de entrada será representada com posições marcadas entre os lexemas:

$$.0 \text{ } lex_1 \text{ } .1 \text{ } lex_2 \text{ } \dots .n-1 \text{ } lex_n \text{ } .n$$

Por exemplo

$$.0 \text{ } A \text{ } .1 \text{ } maior \text{ } .2 \text{ } onda \text{ } .3 \text{ } vinha \text{ } .4 \text{ } a \text{ } .5 \text{ } metros \text{ } .6$$

Algoritmo CYK (II): Tabela de formas sentenciais

Uma tabela M (matriz) $n \times n$ para uma entrada de tamanho n

A posição $M[i, j]$ contém o conjunto de não-terminais que derivam as sub-cadeias da entrada que começam na posição i e terminam na posição j . Isto é:

$$M[i, j] = \{A : A \Rightarrow^* \omega_{i+1} \dots \omega_j\}$$

A matriz é triangular, pois nenhuma subcadeia termina antes do seu início



Cobertura por não-terminais e posições

- Entrada exemplo (em lexemas): 0123456. Onde cada número representa um lexema.
- Em cada $M[i, j]$ leia-se: Conjunto de não terminais que leem começando no lexema da posição i até o lexema da posição j .

	1	2	3	4	5	6
0	01	02	03	04	05	06
1		12	13	14	15	16
2			23	24	25	26
3				34	35	36
4					45	46
5						56

Exemplo de matriz com cobertura

	1	2	3	4	5	6
0	A	A maior	A maior onda	A maior onda vinha	A maior onda vinha a	A maior onda tinha a metros
1		maior	maior onda	maior onda vinha	maior onda vinha a	maior onda vinha a metros
2			onda	onda vinha	onda vinha a	onda vinha a metros
3				vinha	vinha a	vinha a metros
4					a	a metros
5						metros

Usando a matriz para análise explosiva ascendente CYK

	1	2	3	4	5	6
0	A					
1		maior				
2			onda			
3				vinha		
4					a	
5						metros

O → *SujPred*
Suj → *ArtSubs*
Subs → *AdjSubs* | *metros* | *onda* | *vinha*
Pred → *VerbtOD*
OD → *PrepSubs*
Prep → *a*
Verbt → *vinha*
Adj → *maior*
Art → *A*

Usando a matriz para análise explosiva ascendente CYK

Entrada: A maior onde vinha a metros

	1	2	3	4	5	6
0	{Art}					
1		{Adj}				
2			{Subs}			
3				{Verbt, Subs}		
4					{Prep}	
5						{Subs}

O → *Suj Pred*
Suj → *Art Subs*
Subs → *Adj Subs | metros | onda | vinha*
Pred → *Verbt OD*
OD → *Prep Subs*
Prep → *a*
Verbt → *vinha*
Adj → *maior*
Art → *A*

Usando a matriz para análise explosiva ascendente CYK

Entrada: A maior onde vinha a metros

	1	2	3	4	5	6
0	{Art}	{}	{Suj}	{}	{}	{O}
1		{Adj}	{Subs}	{}	{}	{}
2			{Subs}	{}	{}	{}
3				{Verbt, Subs}	{}	{Pred}
4					{Prep}	{OD}
5						{Subs}

O → *Suj Pred*
Suj → *Art Subs*
Subs → *Adj Subs | metros | onda | vinha*
Pred → *Verbt OD*
OD → *Prep Subs*
Prep → *a*
Verbt → *vinha*
Adj → *maior*
Art → *A*

Usando a matriz para análise explosiva ascendente CYK

Ordem de preenchimento da matriz:

Entrada: A maior onde vinha a metros

	1	2	3	4	5	6
0	{ <i>Art</i> } ¹	{ } ³	{ <i>Suj</i> } ⁶	{ } ¹⁰	{ } ¹⁵	{ <i>O</i> } ²¹
1		{ <i>Adj</i> } ²	{ <i>Subs</i> } ⁵	{ } ⁹	{ } ¹⁴	{ } ²⁰
2			{ <i>Subs</i> } ⁴	{ } ⁸	{ } ¹³	{ } ¹⁹
3				{ <i>Verbt</i> , <i>Subs</i> } ⁷	{ } ¹²	{ <i>Pred</i> } ¹⁸
4					{ <i>Prep</i> } ¹¹	{ <i>OD</i> } ¹⁷
5						{ <i>Subs</i> } ¹⁶

O → *Suj Pred*
Suj → *Art Subs*
Subs → *Adj Subs | metros | onda | vinha*
Pred → *Verbt OD*
OD → *Prep Subs*
Prep → *a*
Verbt → *vinha*
Adj → *maior*
Art → *A*

Algoritmo CYK

```

for j=1 to len(entrada):
  PreencheToken(j-1,j)
  for i=j-2 downto 0
    PreencheReducoes(i,j)
  
```

Preenchendo M

$$PreencheToken(j-1, j) = \{N : N \rightarrow lex_j \in P_{gram}\}$$

$$PreencheReducoes(i, j) = \{A : A \rightarrow BC, i < k < j, B \in M[i, k] \text{ e } C \in M[k, j]\}$$

$$M[i, j] = \{\}$$

for $k=i+1$ to $j-1$:

for every $A \rightarrow BC \in P_{gram}$

if $B \in M[i, k]$ and $C \in M[k, j]$ then

$$M[i, j] = M[i, j] \cup \{A\}$$

Como o CYK lida com o não-determinismo da gramática ?



O CYK: Código e complexidade

for $j=1$ to n :

$M[j-1, j] = \{N : N \rightarrow lex_j \in P_{gram}\}$

for $i=j-2$ downto 0

$M[i, j] = \{\}$

for $k=i+1$ to $j-1$:

for every $A \rightarrow BC \in P_{gram}$

if $B \in M[i, k]$ and $C \in M[k, j]$ then

$M[i, j] = M[i, j] \cup \{A\}$

if $S \in M[0, n]$ then ACEITA else REJEITA

CYK efetua n^3 operações de atribuição em um string de tamanho $n = len(entrada)$

$LLC \in \mathcal{O}(n^3)$ em geral e para linguagens determinísticas $\mathcal{O}(n)$



O General Parser de Earley

Earley General Parsing, 1968 PhD Diss e 1970 journal pub, usa uma idéia semelhante sobre uma gramática arbitrária (não precisa ser forma normal de Chomsky) e executa um procedimento de parsing top-down. A complexidade é a mesma do CYK. **Tem complexidade $\mathcal{O}(n^2)$ para gramáticas não-ambíguas.**

Leslie Valiant apresenta em 1975 um General Parser que usa o algoritmo de Strassen para multiplicação de matrizes e um upper-bound de análise sintática geral passa a ser conhecido como $n^{2.81}$.

Como **lower-bound** temos que $\{\omega\omega^R : \omega \in \{a, b\}^*\} \in \Omega(n^2)$, ou seja, **nenhum programa pode reconhecer a linguagem dos palindromos pares em tempo menor que n^2** , onde n é o tamanho do string a ser reconhecido.



OBRIGADO !!!

